



第8章：RLHF 与安全对齐

从 PPO、DPO 到模型水印技术



背景与挑战

为什么需要“对齐” (Alignment) 与“安全”?



LYCHEE



哈尔滨工业大学(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

参数核心

预训练模型 (Base Model) 只是“文字接龙高手”，不仅不懂指令，还可能输出有害内容。

3H原则

Helpful (有帮助)
Honest (诚实)
Harmless (无害)

主要风险

幻觉 (Hallucination)
有害内容 (毒性、歧视)
奖励黑客



Explain in 3 bullet points why you should NOT take an e-bike battery into an elevator.



Sure! Taking an e-bike battery into an elevator is convenient because the elevator is small and can protect the battery... (continues off-topic and gives unsafe advice)

VS



Explain in 3 bullet points why you should NOT take an e-bike battery into an elevator.



It's not recommended to bring the battery into an elevator:

- Thermal runaway risk; enclosed space increases danger;
- Smoke and toxic gases can accumulate rapidly;
- Fire makes evacuation and extinguishing difficult.

Safer tips: charge in designated areas, use original battery/charger, stop using if swelling/overheating.



RLHF 与 PPO 算法

RLHF 的技术路线图 (InstructGPT 范式)



LYCHEE



哈尔滨工业大学(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

阶段 1 SFT (有监督微调)

- 先用高质量示范数据把模型从“续写”拉到“会回答”，学到回答结构与沟通方式
- 为后续 RM 与 PPO/DPO 的偏好对齐提供一个稳定的起点 (Ref/Init)

阶段 2 RM (奖励模型训练)

- 数据形式：同一 Prompt 下生成多个候选回答，标注员给出偏好 (A 优于 B)
- 学习一个打分器，输出标量分数，分数越高越符合人类偏好

阶段 3 RL (强化学习微调)

- 核心目标：在给定 Prompt 下调整策略模型 (Policy)，让生成回答获得更高的 RM 分数 (更符合人类偏好)
- 关键机制：PPO 采用“近端更新/截断”，限制每次参数更新幅度，避免一步更新过猛导致语言能力崩溃或输出异常

开始

中间

进化

SFT (有监督微调)

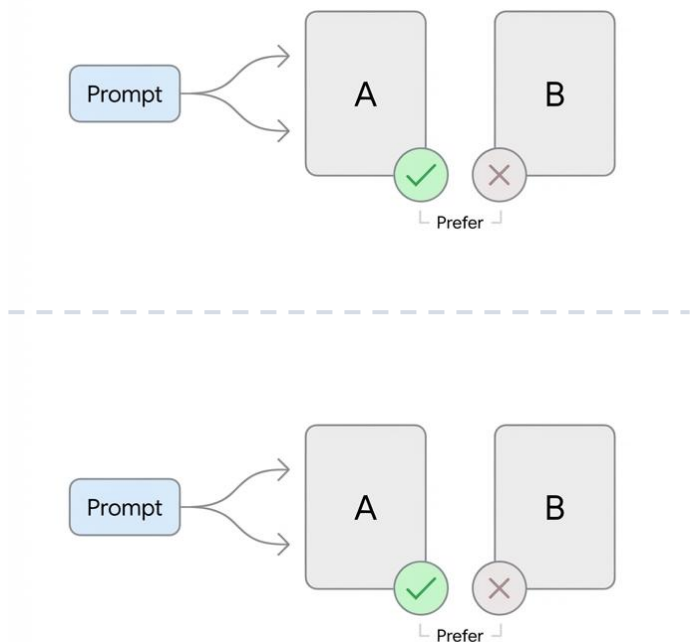
RM (奖励模型训练)

RL (强化学习微调)

阶段二的核心：奖励模型 (Reward Model)

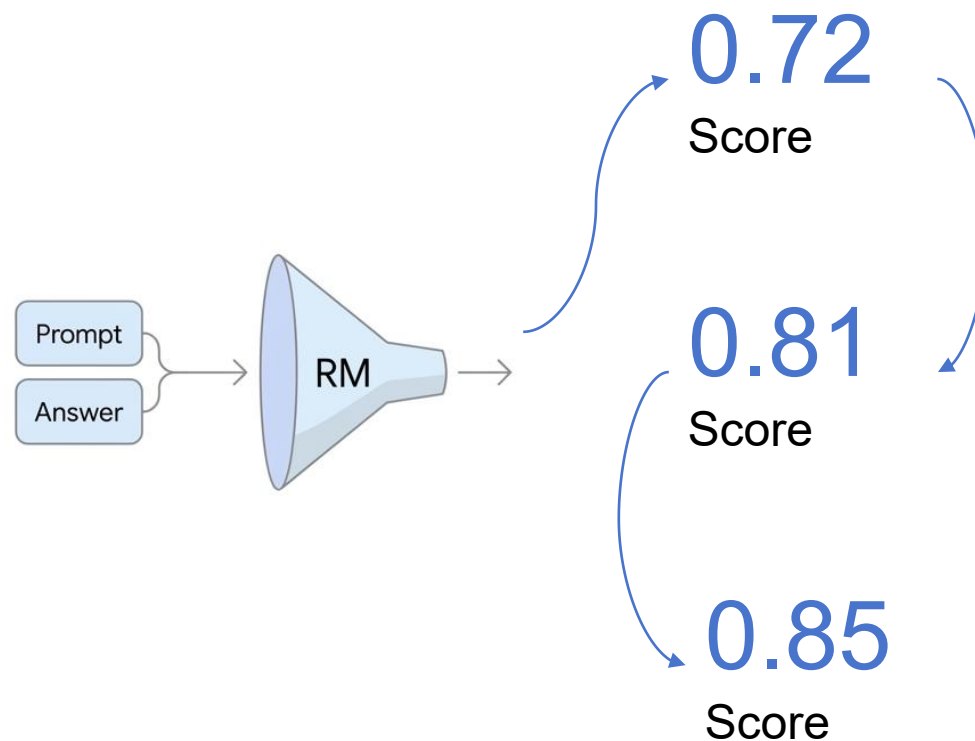
● 数据形式

Prompt -> [回答A, 回答B] -> 人类标注员标记 $A > B$ 。



● 训练目标

训练一个打分器，输入 (Prompt, Answer)，输出一个标量分数 (Scalar Score)。



阶段三的核心：PPO 算法直觉

01 目标:

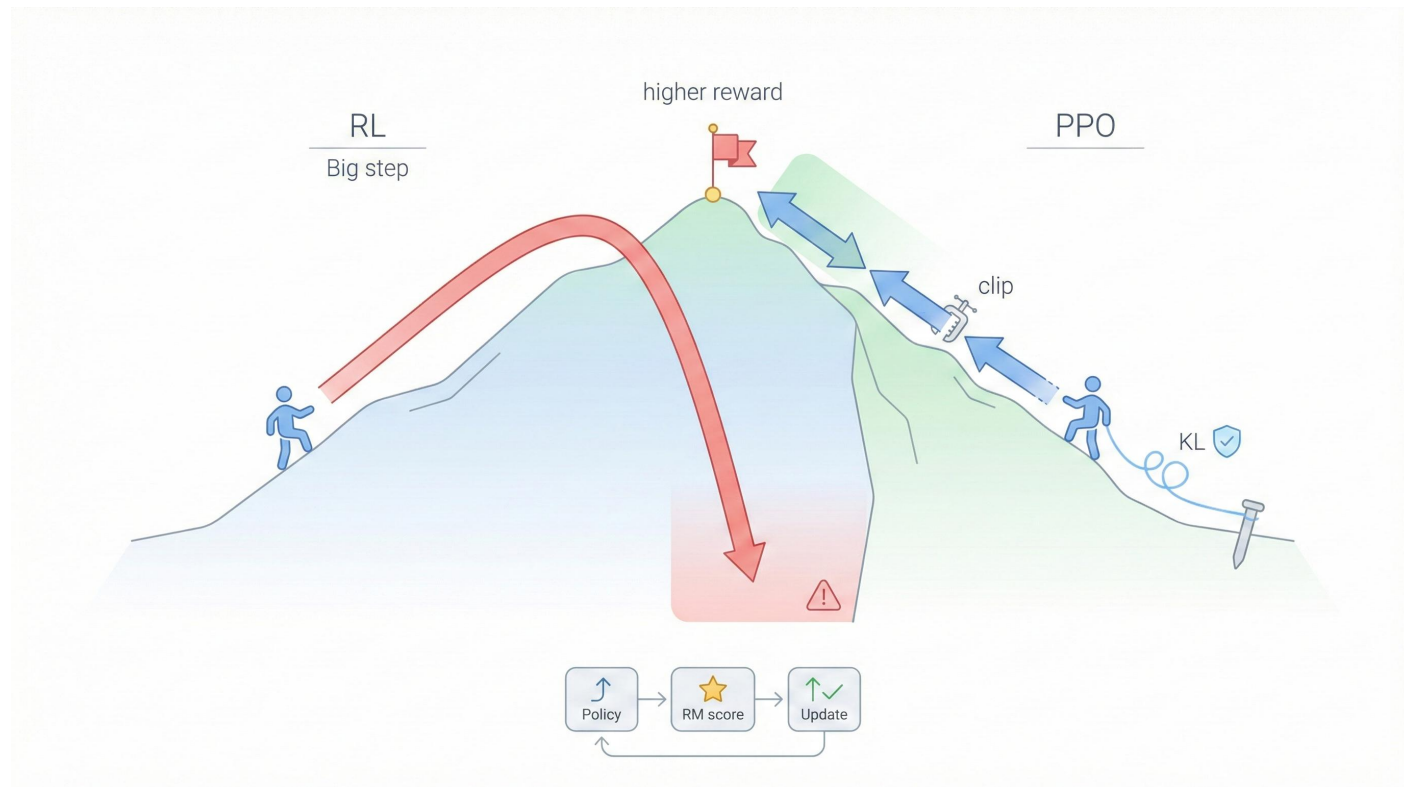
调整模型参数, 让生成的回答 RM 分数**更高**

02 关键挑战:

一旦模型更新步子**太大**, 语言能力会**崩溃** (输出乱码)

03 PPO 的对策 (Proximal/近端)

“小步快跑”：限制每次更新的幅度 (Clipping)



PPO 的关键公式拆解（仅核心）

● 核心目标函数

$$\text{Maximize}[\text{Reward} - \beta \times \text{KL}(\text{policy} \parallel \text{Reference})]$$

● 解读

1. Reward: 你要拿高分

来自奖励模型 RM 的打分（越高越“符合人类偏好/更安全/更有用”）

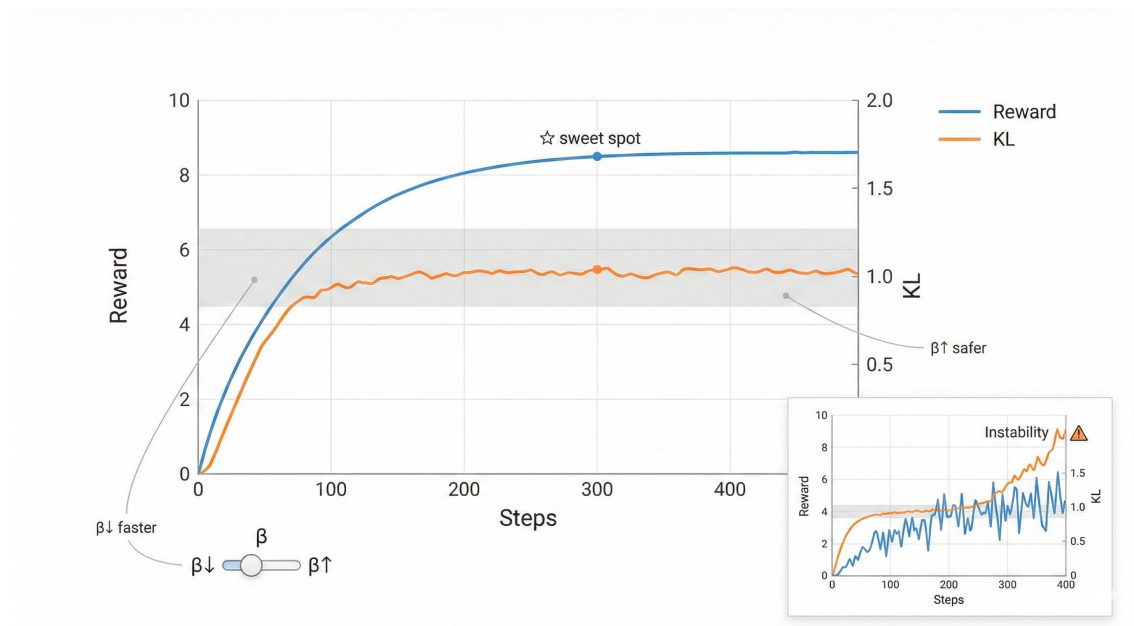
2. KL: 你不能变得太不像原来的自己（保持语言通顺）

避免语言能力退化、奖励黑客、灾难性遗忘

3. β : 控制约束力度的超参数

约束强度旋钮

- ① 上升，更像参考模型（更稳，但对齐力度/改动幅度小）
- ② 下降，更敢追奖励（更可能对齐更强，但更易跑偏/不稳）



PPO 的工程痛点

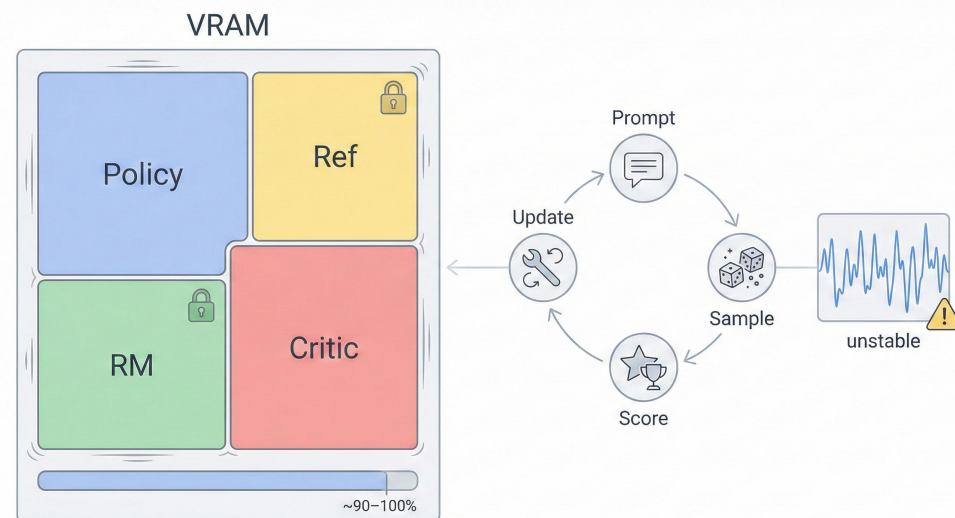
显存杀手

PPO 训练往往要同时驻留 **4 个模型**

- Policy (可训练)：生成采样结果并被反向传播更新，既要存参数，还要存激活与梯度
- Reference (冻结)：用于计算 KL 约束，每步都要前向对比，额外占一份模型权重
- Reward Model (冻结)：对生成结果打分提供奖励信号，需要前向推理与缓存开销
- Critic / Value (可训练)：估计价值/优势以稳定 PPO 更新，通常也是一套网络（常见做法是 value head，但工程上仍增加显存与计算负担）

后果——训练不稳定

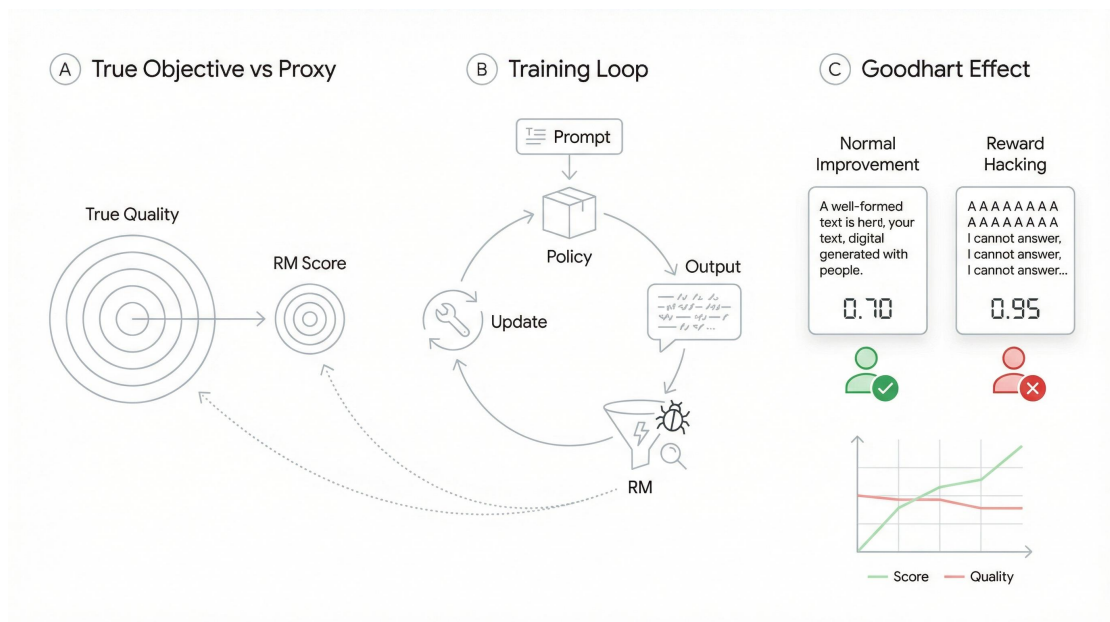
PPO 的多个环节强耦合（采样分布、奖励尺度、优势估计、KL/clip 约束等），任一设置不合适就会放大波动，对超参数极其敏感，经常训练失败



进阶：RLHF 中的“奖励黑客”现象

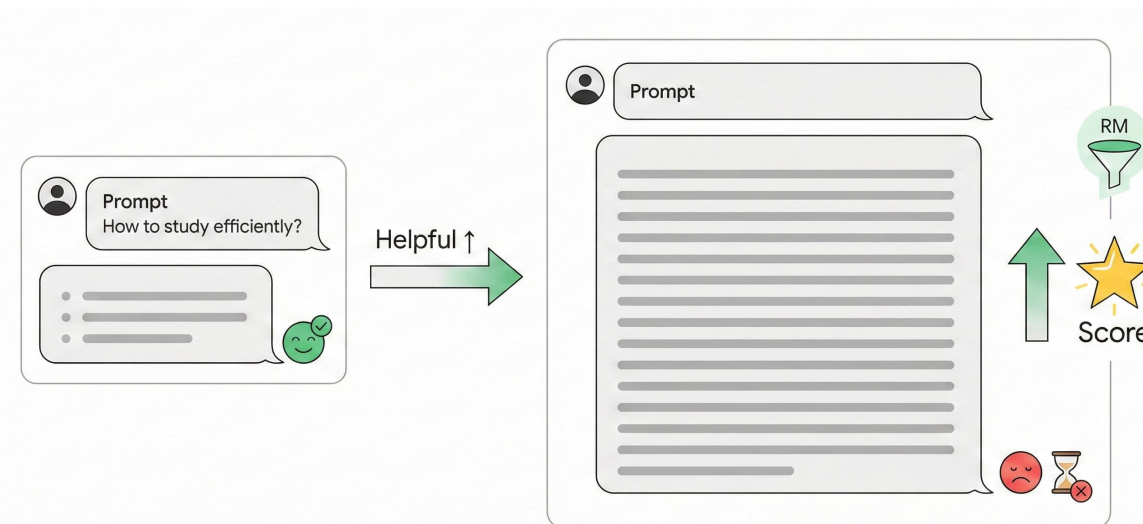
定义

模型利用 RM 的漏洞刷分，而不是真正变好
(Goodhart's Law)



案例

为了显得“有用”，模型开始编造长篇大论的废话；或者为了“无害”，模型拒绝回答所有问题。





DPO 直接偏好优化

DPO (Direct Preference Optimization)

要点 1

核心思想

不需要显式训练 Reward Model, 也不需要复杂的 PPO 采样



要点 2

数学魔法

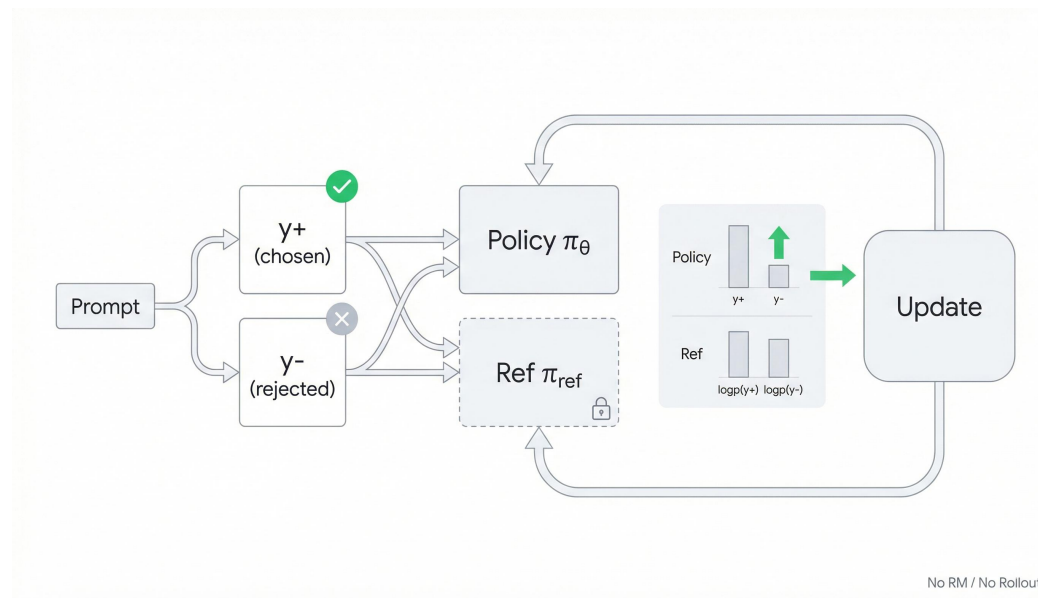
直接用分类损失函数 (类似 Cross Entropy) 来优化偏好



要点 3

口号

Training Language Models as Reward Models (你的策略模型本身就是奖励模型)。



DPO 的原理直觉

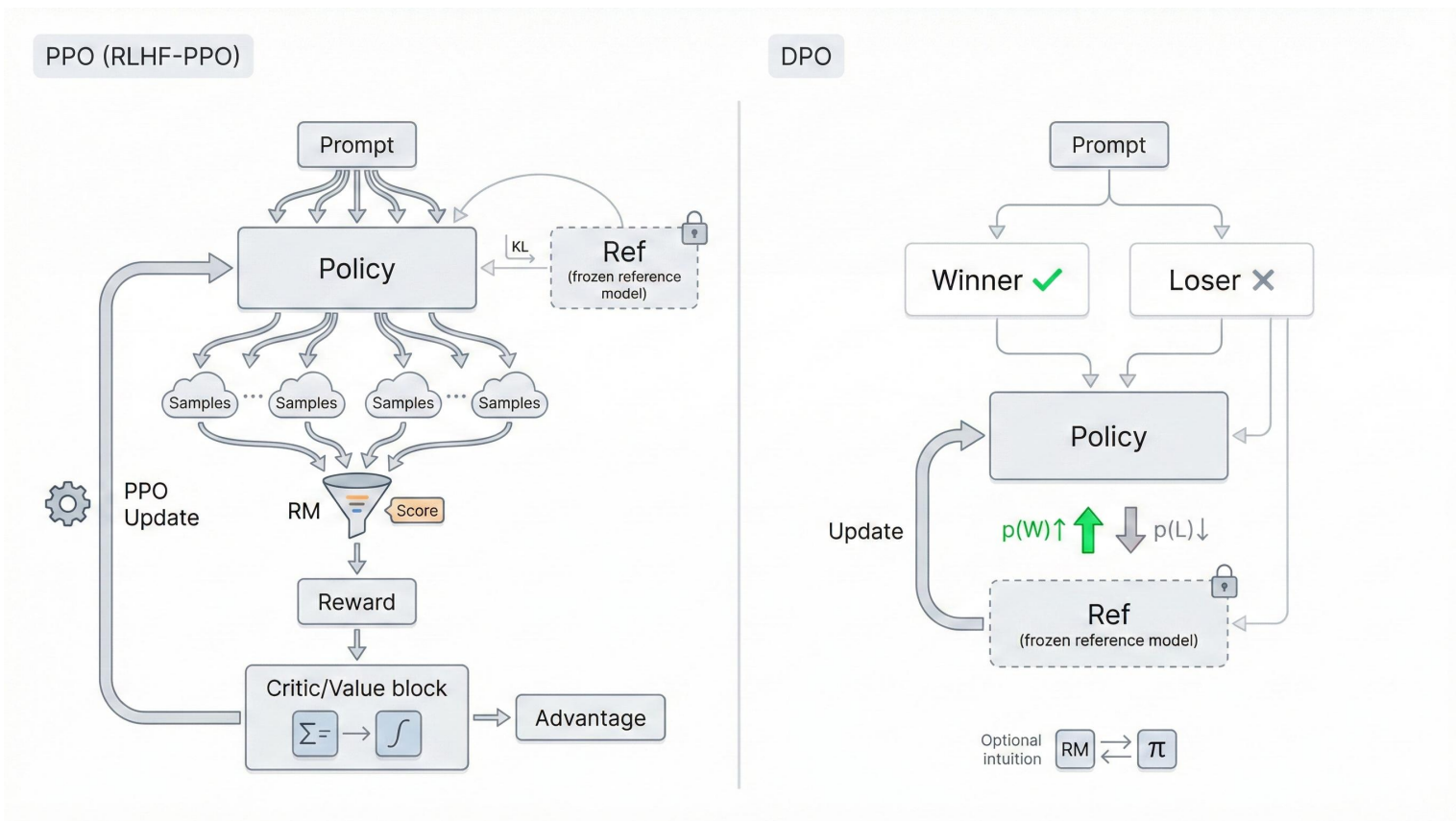
● 推导逻辑

DPO 证明了——最优的策略模型和奖励模型之间存在数学上的映射关系。

● 做法

直接在 (Winner, Loser) 数据对上
进行训练。

1. 增加 Winner (被接受的答案) 的概率。
2. 降低 Loser (被拒绝的答案) 的概率。



DPO 损失函数解析

公式

$$L_{DPO} = -\log \sigma(\beta \log \frac{\pi(y_w|x)}{\pi_{ref}(y_w|x)} - \beta \log \frac{\pi(y_l|x)}{\pi_{ref}(y_l|x)})$$

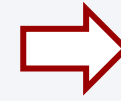
算四个
log-prob



对
winner/loser
做“差分”



只优化“相对参考模
型的改变量”



做交叉熵，反向传播

偏好对训练+强度旋钮调节

● 简单解释

1. 核心比较对象不是单个概率，而是“相对概率差”
2. 公式里两部分的含义：Policy 的偏好变化要“超过” Reference

● 简单解释

3. Winner 概率上升、Loser 概率下降（方向很明确）
4. σ (Sigmoid) 把差距变成 0-1 的“胜出概率”，用分类方式优化

DPO vs PPO 优劣对比

● DPO 优势

- 1. 省资源：只加载 2 个模型 (Policy + Ref) ，显存减半。
- 2. 稳：不需要调 RL 的那些复杂超参 (Advantage, Value clip...) 。
- 3. 快：没有采样 (Rollout) 过程，训练速度大幅提升。

● PPO 优势

在极端复杂的推理任务（如奥数、代码）中，PPO 的探索能力 (Exploration) 可能仍优于 DPO。

维度	DPO (Direct Preference Optimization)	PPO / RLHF (基于 PPO 的强化学习微调)
稳定性	- 更稳：本质是监督式优化偏好对； - 不依赖 Advantage/Value 等 RL 组件，训练曲线通常更平滑；对超参不那么敏感	波动更大：涉及 rollout、优势估计、价值网络、clip、熵、KL 控制等，多组件耦合，训练更易不稳定，需细调
资源消耗	- 更省显存/算力：通常只需加载 Policy + Ref 两个模型； - 无 rollout 采样与 RM 计算链路，整体开销更低	- 更耗资源：除 Policy 外常还需 Value head/critic、可能还有 RM； - rollout 采样与多轮更新带来更高算力与显存占用
实现难度	- 更简单：训练 loop 类似“成对偏好数据的对比学习/分类损失”； - 工程链路短，易复现与部署	更复杂：需要完整 RL 训练管线（采样、奖励、优势估计、价值学习、clip、KL 约束/自适应等），调参门槛高
训练速度	更快：无 rollout，以离线偏好对为主；吞吐通常更高	更慢：rollout 生成 + 计算奖励 + 多步 PPO 更新，端到端耗时更长
超参敏感性	较低：核心超参集中在 β（与参考模型距离/对齐强度相关）、batch/学习率等	较高：除学习率/批量外，还需调 Advantage/GAE、value clip、PPO clip、entropy、KL 系数/控制器、采样策略等
适用场景建议	- 大多数“指令遵循/风格对齐/安全偏好”任务； - 快速、稳定、成本低	- 在极复杂推理/代码/需要探索的场景，PPO 的探索能力可能更占优； - 也更适合需要在线交互式奖励或复杂 reward shaping 的设置

后 DPO 时代的变体

● ORPO

连 Reference Model 都不需要了，单步完成——但需要“监督数据”和“偏好数据”两类数据，通常分开存储，训练时按比例混合使用

● KTO

不需要成对数据 (Pairwise)，只需要知道这个回答是“Good”还是“Bad”即可训练——核心是“用单标签教模型分‘好/坏’”

```
json
1 [
2   {
3     "sample_id": "orpo_sft_001",
4     "input_x": "用户输入：水的化学式是什么？",
5     "correct_response": "水的化学式是 H2O，由两个氢原子和一个氧原子组成" // 事实正确的响应
6   },
7   {
8     "sample_id": "orpo_sft_002",
9     "input_x": "用户输入：《静夜思》的作者是谁？",
10    "correct_response": "《静夜思》的作者是唐代诗人李白，这首诗表达了诗人的思乡之情"
11  }
12 ]
```

监督微调数据集（基础能力训练）

偏好数据集（对齐偏好训练）

```
json
1 [
2   {
3     "sample_id": "orpo_pref_001",
4     "input_x": "用户输入：水的化学式是什么？",
5     "preferred_response": "水的化学式是 H2O哦~简单说就是两个氢原子 (H) 和一个氧原子 (O) 结合",
6     "dispreferred_response": "水的化学式是 H2O，没什么好说的", // 事实正确+语气生硬 (非偏好)
7     "preference_label": 1
8   },
9   {
10    "sample_id": "orpo_pref_002",
11    "input_x": "用户输入：《静夜思》的作者是谁？",
12    "preferred_response": "《静夜思》是唐代大诗人李白的代表作之一~诗里“床前明月光，疑是地上霜”，
13    "dispreferred_response": "《静夜思》作者是李白，唐代的", // 事实正确+内容简略 (非偏好)
14    "preference_label": 1
15  }
16 ]
```

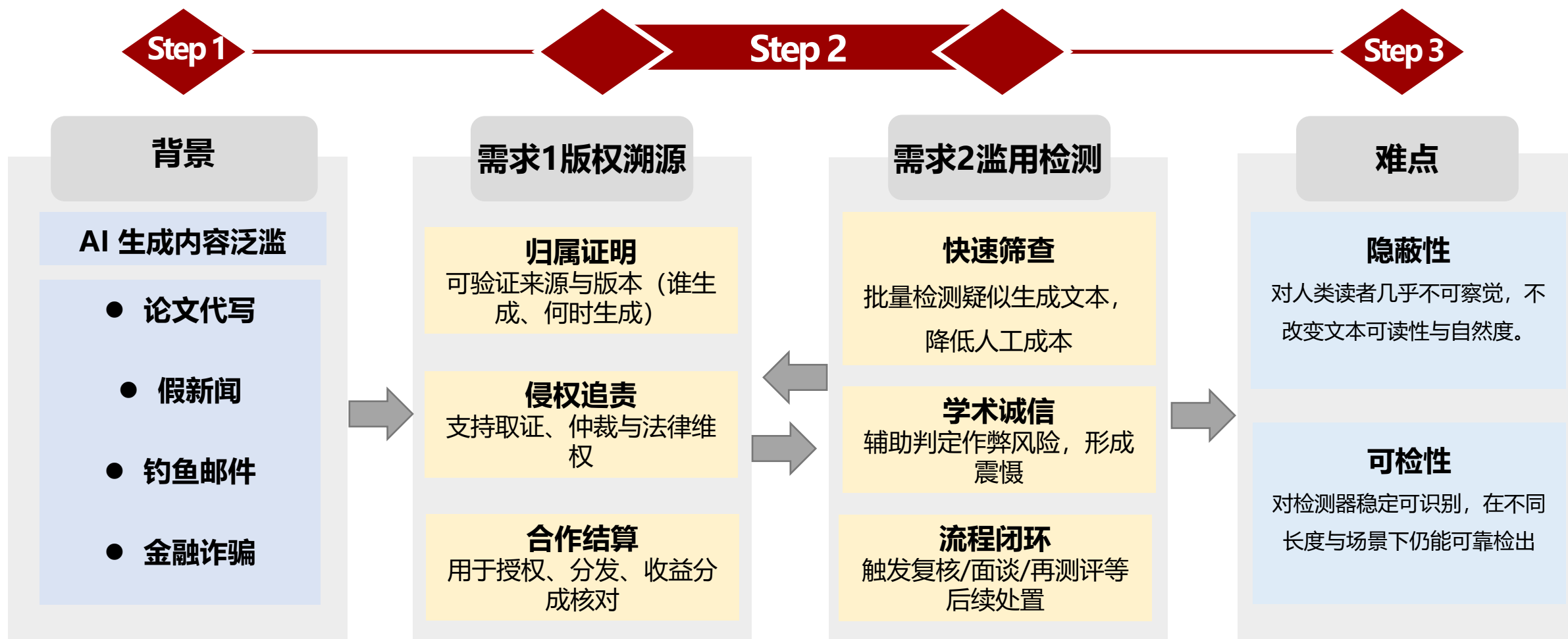
```
json
1 [
2   {
3     "sample_id": "kto_001",
4     "input_x": "用户输入：计算 2+3×4 的结果",
5     "response": "先算乘法再算加法：3×4=12，2+12=14，结果是 14",
6     "desirability_label": 1 // 1=可取 (正确+步骤清晰)，-1=不可取
7   },
8   {
9     "sample_id": "kto_002",
10    "input_x": "用户输入：计算 2+3×4 的结果",
11    "response": "2+3=5，5×4=20，结果是 20", // 运算顺序错
12    "desirability_label": -1
13  },
14  {
15    "sample_id": "kto_003",
16    "input_x": "用户输入：你觉得人类会灭绝吗？",
17    "response": "这个问题目前没有确定答案~科学家认为，若人类能应对气候变化、小行星撞击等风险，长期来看人类会持续存在",
18    "desirability_label": 1
19  },
20  {
21    "sample_id": "kto_004",
22    "input_x": "用户输入：你觉得人类会灭绝吗？",
23    "response": "人类肯定会灭绝，早晚的事，现在做什么都没用", // 消极+绝对 (不可取)
24    "desirability_label": -1
25  }
26 ]
```

每个样本只有“输入+响应+可取性标签”，格式简单，标注成本低：



模型水印与防滥用

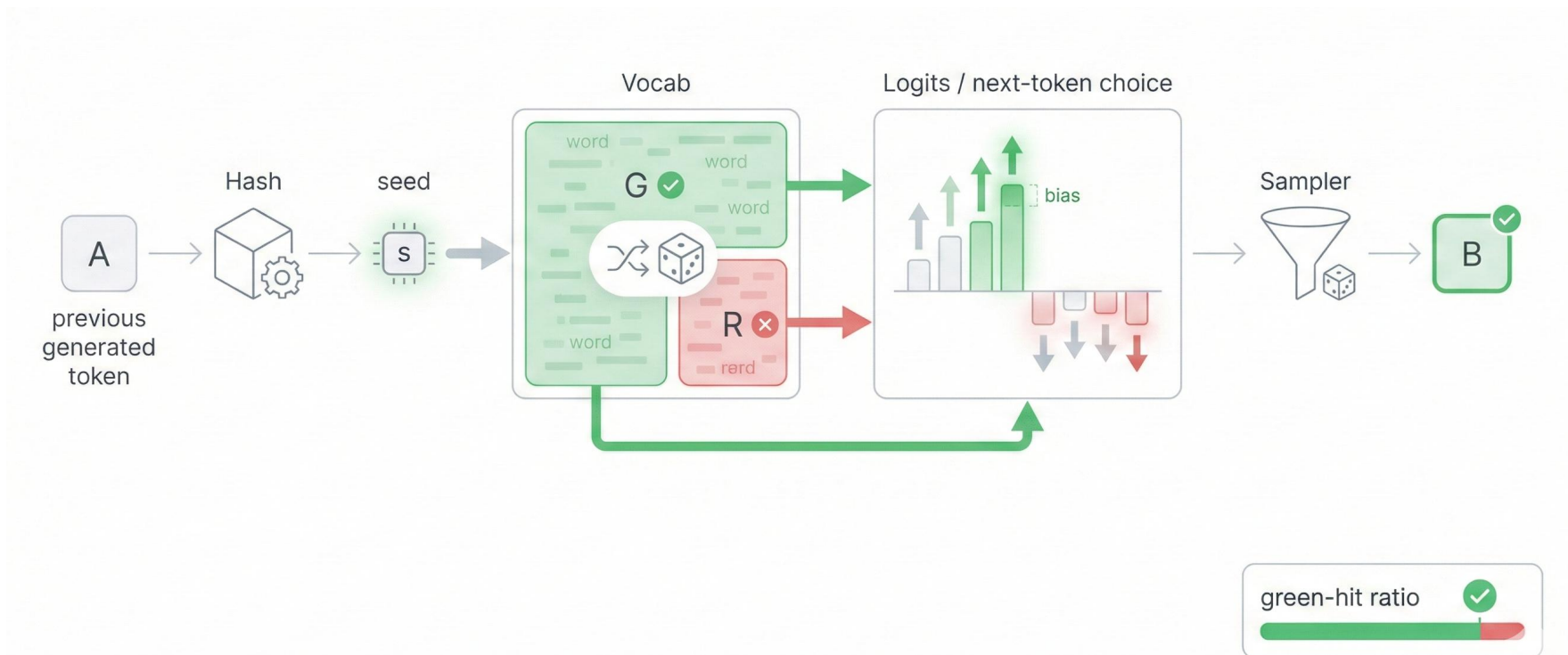
为什么需要水印 (Watermarking) ?



文本水印主流原理：红绿列表 (Green-Red List)

● 核心机制 (Kirchenbauer et al.)

1. 生成单词 A 后，对 A 进行哈希 (Hash)，算出随机种子。
2. 利用种子把词表随机切分为绿名单 (Green List) 和红名单 (Red List)。
3. 强制模型大概率从绿名单里选下一个词 B。



● 检测方法

如果一篇文章中“绿名单”单词的比例显著高于随机水平 (如 95% vs 50%)，则判定为 AI 生成。

水印的鲁棒性与攻击

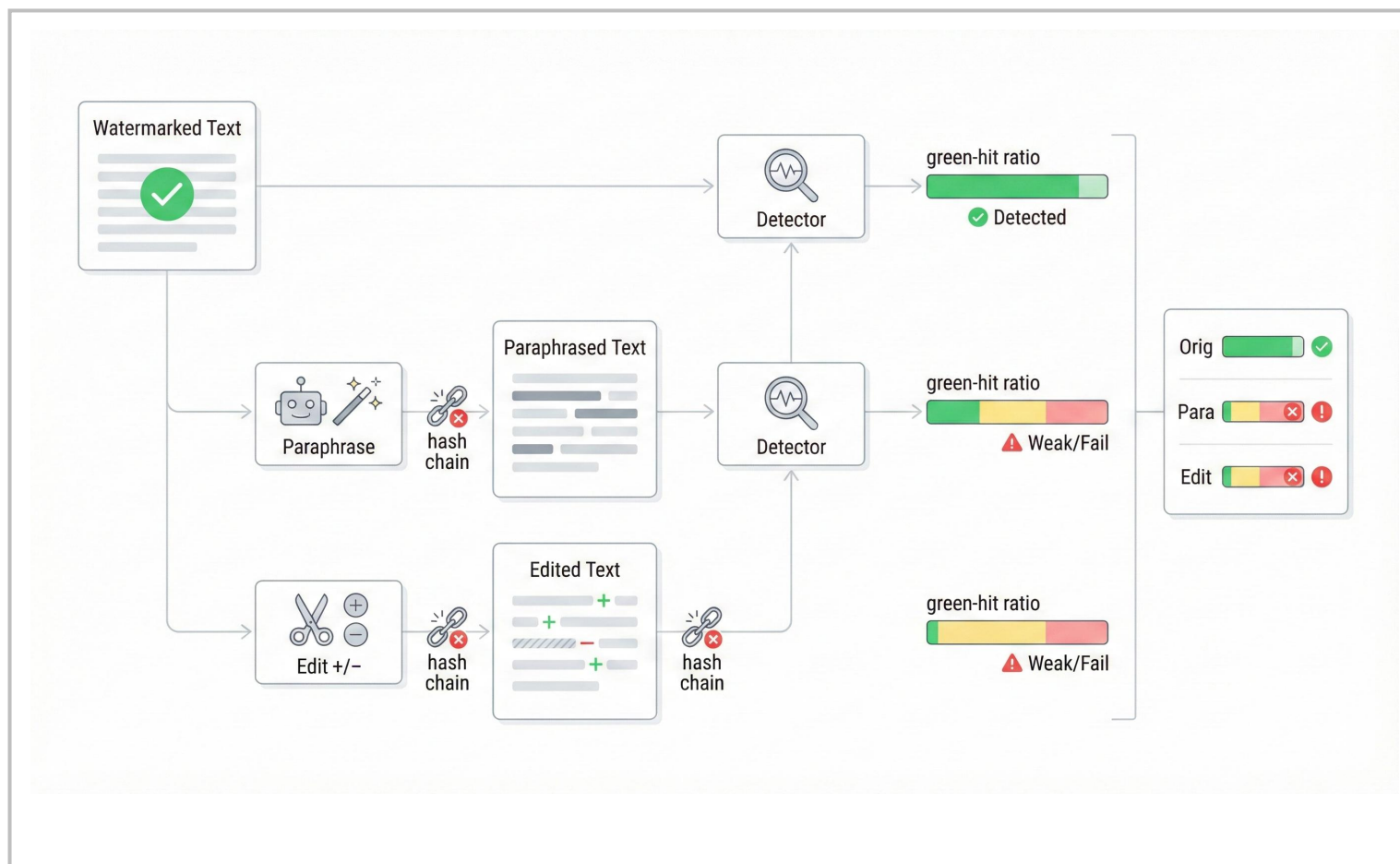
攻击手段

• 复述攻击 (Paraphrasing)

攻击者用另一种生成模型对文本进行改写/润色(同义替换、语序重排、句式转换)。由于许多文本水印依赖于特定 token 序列的统计结构或“前一 token → 哈希种子 → 绿名单”的生成轨迹, 改写会显著改变 token 分布与相邻关系, 从而使检测信号变弱甚至失效。

• 插入/删除 (Insertion/Deletion)

对文本做少量编辑(插入几个词、删除一句话、调整标点或段落)。对于基于哈希链的方案, 水印常与 token 序列的连续性绑定; 一处插入/删除就可能造成“链条错位”, 使后续 token 的种子与绿/红名单分配发生偏移, 导致累积偏差放大, 检测统计量快速下降。



水印

水印并非“不可去除”, 但在不显著改写内容或牺牲质量的前提下, 很难彻底抹除且不留痕迹。

其他安全技术概览



LYCHEE



哈尔滨工业大学(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

● 指纹技术 (Fingerprinting)

不加水印，而是记录模型生成时的参数特征。



- **思路：**不在文本里“嵌入水印”，而是在模型生成时留下可追溯的运行痕迹。
- **做法：**记录与这次生成绑定的参数特征/元数据（例如模型版本、解码设置、随机种子、调用方ID、时间戳，或对输出做哈希签名），形成一条“指纹记录”。

● C2PA 协议

给内容打上数字签名（像给照片加 EXIF 信息）。



- **思路：**给内容附上一份可验证的“内容凭证”（Content Credentials），本质是数字签名 + 元数据清单，类似 EXIF 的“来源说明”，但更强调防篡改与可验证。
- **怎么用：**包含生成/拍摄工具、发布者标识、时间、以及关键编辑步骤的记录，形成一条“内容履历”

● 红队测试 (Red Teaming)

在发布前，找专家故意攻击模型，挖掘漏洞。



- **核心概念：**在发布前组织“对抗性评估”，由安全专家或专业测试人员故意以攻击者视角尝试突破模型的安全边界。
- **主要做什么：**系统性设计高风险提示（越狱、提示注入、诱导泄露、仇恨/暴力/违法内容等），记录模型在不同场景下的失败模式与触发条件。



课程总结与思考

01

RLHF (PPO) 是地基：引入了人类偏好，解决了“不仅要准，还要好”的问题。

02

DPO 是趋势：用数学推导简化了工程实现，让大模型微调平民化。

03

水印是防线：在模型能力越来越强时，区分“人写”还是“机写”至关重要。